

can you print scheduler runqueue list of processor

If you're asking for the scheduler run queue of a processor on Linux, there are several ways to inspect it depending on the level of detail you need.

Current Run Queue Length

</> Bash

```
cat /proc/loadavg
```



Example:

```
0.25 0.30 0.28 2/845 12345
```



The `2/845` means:

- `2` tasks are runnable (running or waiting for CPU)
- `845` total tasks in the system

Per-CPU Scheduler Run Queues

</> Bash

```
grep . /proc/schedstat
```



or

```
</> Bash
```



```
cat /proc/sched_debug
```

The scheduler debug file contains entries such as:

```
cpu#0
```



```
.nr_running          : 3  
.nr_switches         : 1234567
```

where `nr_running` is the number of runnable tasks on that CPU's run queue.

To view only run queue lengths:

```
</> Bash
```



```
grep -A2 "cpu#" /proc/sched_debug | grep nr_running
```

Real-Time Monitoring

```
</> Bash
```



```
watch -n1 'grep nr_running /proc/sched_debug'
```

Using vmstat

```
</> Bash
```



```
vmstat 1
```

Output:

```
procs  -----memory-----  ---swap--  -----io-----  -system--  -----cpu-----  
r  b  
3  0
```



r = number of runnable processes waiting for CPU.

Using eBPF (Modern Kernels)

If you have BCC tools installed:

```
</> Bash
```



```
runqlen
```

or

```
</> Bash
```



```
runqlat
```

These provide detailed per-CPU run queue statistics and scheduling latency.

Kernel-Level Inspection

For a specific CPU:

 **Bash**



```
cat /proc/sched_debug | sed -n '/cpu#0/,/^$/p'
```

This prints the run queue contents and scheduler state for CPU 0.

If you tell me:

- Linux distribution
- Kernel version
- Whether you want the **queue length** or the **actual list of tasks in each CPU run queue**

I can provide the exact command.

I want the list of process id(s) waiting on a cpu queue

Linux does **not expose the exact contents of each CPU's CFS run queue** (the list of PIDs waiting on a particular CPU) through a stable `/proc` interface. The scheduler maintains these queues internally in kernel memory.

You have a few options:

1. See runnable tasks (system-wide)

</> Bash



```
ps -eLo pid,tid,psr,stat,comm | grep '^.* R '
```

- **R** = runnable or running.
- **psr** = CPU the task was last scheduled on.

This shows tasks that are eligible to run, but not their exact position in a CPU run queue.

2. Use `/proc/sched_debug`

</> Bash



```
sudo cat /proc/sched_debug
```

On some kernel versions, each CPU section contains runnable-task information such as:

runnable tasks:

```
S          task  PID
...
```



or similar scheduler diagnostics.

Search for:

</> Bash



```
sudo grep -A50 "runnable tasks" /proc/sched_debug
```

However, the format varies significantly across kernel versions, and many distributions disable detailed task listings.

3. Trace scheduler events with eBPF (best method)

Using **bpfttrace**:

```
</> Bash   
  
sudo bpfttrace -e '  
tracepoint:sched:sched_wakeup  
{  
    printf("WAKEUP pid=%d comm=%s cpu=%d\n",  
          args->pid, args->comm, cpu);  
}  
'
```

Or monitor enqueue/dequeue activity.

With BCC tools:

```
</> Bash   
  
runqlen  
runqlat  
offcputime
```

These reveal which tasks are waiting and for how long.

4. Kernel debugging

If you need the **actual run queue contents per CPU**, the most reliable method is a kernel module or eBPF program that inspects:

```
</> C  
  
struct rq  
struct cfs_rq
```



and traverses:

```
</> C  
  
rq->cfs_tasks
```



(or the kernel-version-specific equivalent).

For example, on modern kernels the runnable tasks are stored in a red-black tree (`cfs_rq.tasks_timeline`) rather than a simple linked list.

Example: Obtain runnable PIDs from sched_debug

If your kernel exposes them:

Ask ChatGPT

Start writing

```
</> Bash  
  
sudo awk '  
/cpu#/ {cpu=$1}  
/runnable tasks:/ {show=1; next}  
/^$/ {show=0}
```



```
show {print cpu, $0}  
' /proc/sched_debug
```

Could you tell me:

- Kernel version (`uname -r`)
- Distribution (Ubuntu, RHEL, Debian, etc.)
- Whether you want **all runnable PIDs** or **PIDs queued on a specific CPU (e.g., CPU 7)**?

The answer depends heavily on the scheduler implementation in that kernel version.